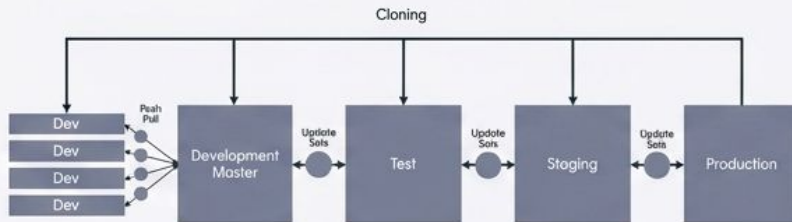


Legacy: Environment Development

Mechanism: Update sets deployed down an instance chain.

Pain Points: Time-consuming, manual collision resolution, requires instance cloning.

Limitation: Hard to isolate parallel development.

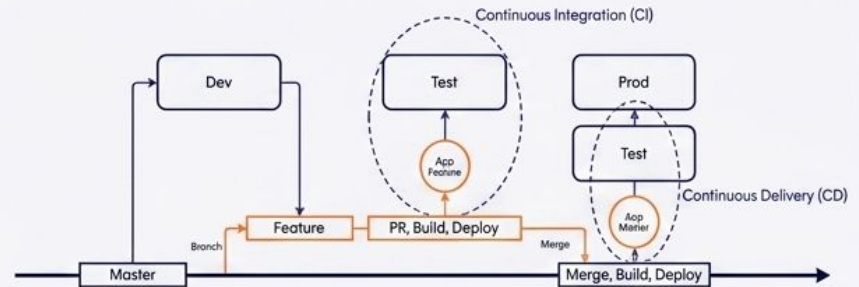


Modern: Full Stack Development

Mechanism: Source control and GitHub Flow.

Strengths: Better code isolation, leverage CI/CD pipelines.

Advantage: Master/Main branch is the definitive production version.



Why DevOps Was Needed

Traditional Software Development Challenges



Development Team

- Focused on writing code
- Delivered apps quickly



Operations Team

- Focused on stability
- Preferred fewer changes



Common Problems

- Slow releases & manual deploys
- Environment mismatch
- Production issues & siloed communication

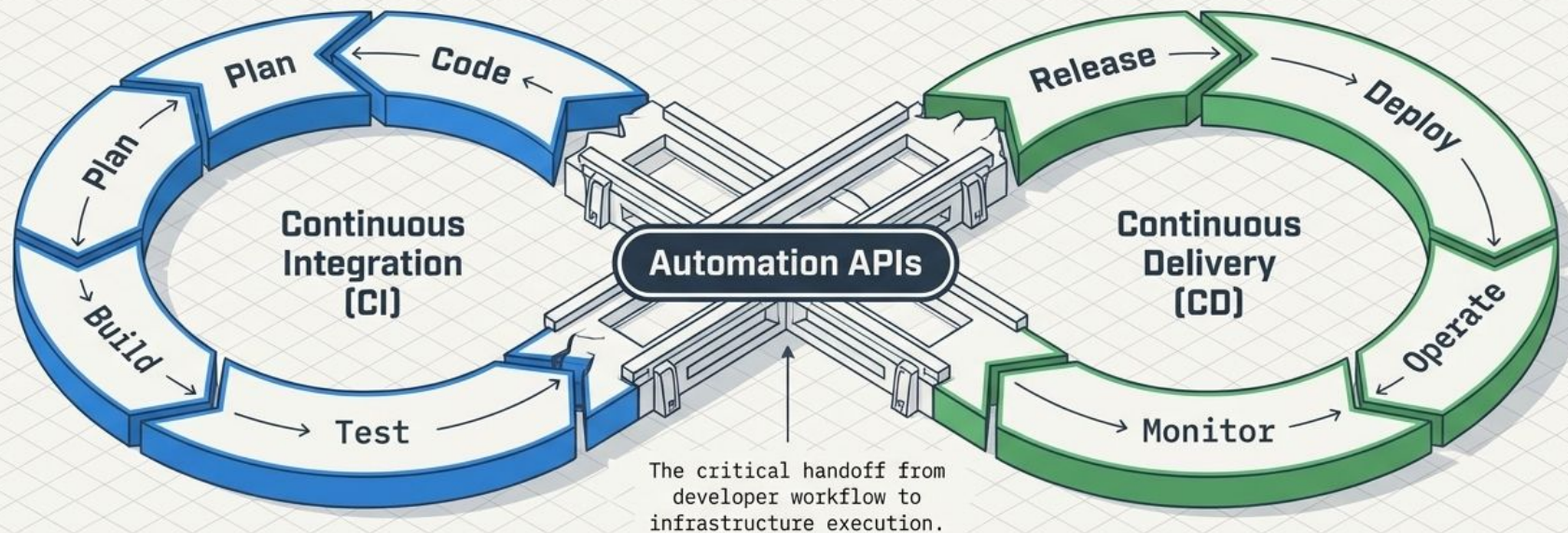
The Real-World Conflict

Dev: "Application works on my machine."

Ops: "Application fails in production."

This gap between teams led to the birth of DevOps.

Exploding the CI/CD Engine



Mechanic: Automated build-and-test steps ensure repository merges are highly reliable.

Mechanic: Seamless, rapid delivery of incremental code changes to production.

Traditional vs. DevOps Model

Comparative Workflow Analysis

Traditional Model

WORKFLOW

Requirement → Development → Testing → Operations → Customer

ISSUES

- Siloed teams
- Slow feedback
- Longer release cycles
- Higher failure rate

DevOps Model

WORKFLOW

Req → Dev → Test → Deploy → Monitor → Feedback

BENEFITS

- Shared responsibility
- Faster feedback
- Continuous delivery
- Better customer experience



Features	DevOps	Agile
Agility	Agility in both Development & Operations	Agility in only Development
Processes/ Practices	Involves processes such as CI, CD, CT, etc.	Involves practices such as Agile Scrum, Agile Kanban, etc.
Key Focus Area	Timeliness & quality have equal priority	Timeliness is the main priority
Release Cycles/ Development Sprints	Smaller release cycles with immediate feedback	Smaller release cycles
Source of Feedback	Feedback is from self (Monitoring tools)	Feedback is from customers
Scope of Work	Agility & need for Automation	Agility only

DevOps Defined: Culture & Function

- A collaborative **CULTURE** between Development and Operations teams,
- promoting **faster**, **automated**, and **repeatable** code deployment to production.



Collaboration

Fusion

The critical handoff from developer workflow to infrastructure execution.

- between Development and
- The term 'DevOps' is a functional fusion, a combination of two words: Development and Operations.



Mechanic: Automated build-and-test steps ensure repository merges are highly reliable.

Mechanic: Seamless, rapid delivery of incremental code changes to production.

Core Principles of DevOps

Key pillars for modern software delivery



1. Collaboration

Development and Operations work together as a unified team.



2. Automation

Automate repetitive tasks to increase speed and reliability.

EXAMPLES:

- Code build
- Testing
- Deployment
- Infra provisioning



3. Continuous Integration

Developers frequently merge code to catch issues early.



4. Continuous Delivery

Applications are always in a deployment-ready state.



5. Monitoring

Continuous visibility into application health and performance.

DevOps Lifecycle

Stages of DevOps: From Planning to Monitoring

Plan

- Requirement gathering
- Sprint planning
- Backlog creation

Develop

- Coding
- Version control

Build

- Compile source code
- Package application

Test

- Unit testing
- Integration testing

Release

- Version management

Deploy

- Deploy to environments

Operate

- Manage production systems

Monitor

- Collect logs and metrics

DevOps Lifecycle: Real-World Example

Case Study: Online Shopping Website - "Add to Cart" Feature

Plan

New "Add to Cart" feature requested by product owners.

Develop

Developers write and commit the feature code.

Build

Source code is compiled into a deployable application package.

Test

QA validates functionality against requirements.

Deploy

The application is deployed to the production environment.

Operate

Live users start using the new feature on the website.

Monitor

System performance and user errors are tracked in real-time.

Feedback

User insights lead to improvements in the next development cycle.

Continuous Cycle: Monitoring & Feedback trigger the next Planning phase.

DevOps Culture

DevOps is More Than Tools

People

- Shared ownership
- Collaboration

Process

- Agile methodologies
- Continuous feedback

Technology

- Automation
- Cloud platforms

CALMS Framework

Culture **A**utomation **L**ean **M**easurement **S**haring

The foundational values that drive DevOps success.

DevOps Toolchain

Modern tools for every stage of the lifecycle



Planning

- Jira
- Azure Boards



SCM

- GitHub
- GitLab



CI/CD

- Jenkins
- GitLab CI/CD



Containers

- Docker



Orchestration

- Kubernetes



Monitoring

- Prometheus
- Grafana

CI/CD in DevOps

Automating the Path from Code to Production



Continuous Integration

Developers frequently merge code into a shared repository.

Benefits:

- Early bug detection
- Faster feedback
- Better code quality



Continuous Delivery

Every successful build is ready for deployment.

Requires manual approval for the final release step.



Continuous Deployment

Every successful build is automatically deployed.

Fully automated end-to-end process without manual gates.

Pipeline Flow

Code → Build → Test → Deploy

Infrastructure as Code (IaC)

Managing infrastructure through software instead of manual processes

Traditional Approach

- Manual server creation
- Manual configuration scripts & steps

IaC Approach

- Automated provisioning
- Version-controlled infrastructure templates

Popular Tools

 Terraform

 AWS CloudFormation

 Ansible

Key Benefits

Consistency: Eliminate "drift" by ensuring all environments are identical.

Repeatability: Rapidly spin up testing or production environments.

Speed: Reduce provisioning time from days to minutes through automation.

Benefits of DevOps

Business Benefits

- Faster delivery
- Reduced costs
- Improved customer satisfaction
- Competitive advantage

Technical Benefits

- Automated deployments
- Faster recovery
- Better scalability
- Improved security

Team Benefits

- Better communication
- Shared responsibility
- Reduced operational friction

DevOps Engineer: Roles & Responsibilities

Daily Responsibilities

Source Control

Manage code repositories.

Cloud & Infrastructure

Provision and maintain infrastructure.

Cross-Functional Collaboration

Working with teams:

- Developers
- QA / Testing
- Security
- Operations

CI/CD

Create and maintain pipelines.

Monitoring & Health

Track application health & metrics.

Containers & K8s

Build Docker images & manage apps.

Security & DevSecOps

Implement best security practices.

Typical DevOps Workflow

Developer → Git → CI/CD → Docker → Kubernetes → Cloud → Monitoring → Users

The Legacy Bottleneck: Environment-Based Development

